

	Curso Superior em Análise e Desenvolvimento de Sistemas
	Disciplina: Padrões de Projeto, Frameworks e API
	Profª. Denise do Rocio Maciel
	LISTA DE EXERCÍCIOS: LISTA DE EXERCÍCIO DESIGN PATTERN - ETAPA 2

FCC – 2026 – TJ-CE – Analista Judiciário - Área Técnico Administrativa

1. Uma equipe de arquitetura de software de um Tribunal de Justiça iniciou a modernização incremental de um sistema processual monolítico. A estratégia prevê que determinadas capacidades do sistema legado sejam substituídas por novos serviços, sem interrupção global da operação. Considerando o padrão Strangler Fig, a conduta tecnicamente coerente é

- a) concluir os serviços planejados antes da troca operacional, reduzindo o período de convivência entre as arquiteturas antiga e nova.
- b) duplicar as funções do legado em serviços equivalentes e permitir que cada unidade usuária defina qual versão utilizará.
- c) iniciar a troca por bibliotecas e componentes técnicos comuns, mantendo as regras de negócio no monólito até a etapa final.
- d) selecionar uma capacidade do legado, implementar sua nova versão em serviço separado e encaminhar gradualmente o tráfego correspondente para essa implementação.
- e) preservar as funções existentes no monólito e usar os novos serviços para requisitos ainda não atendidos pela aplicação original.

FCC – 2026 – MPE-AL – Analista do Ministério Público

2. Para desenvolver um módulo de notificações processuais conforme o perfil do réu, o sistema deve atender aos seguintes requisitos: I. Instanciar diferentes tipos de alertas (e-mail, SMS, Push), permitindo a inclusão de novos tipos sem alteração do código existente; II. Garantir que o registrador de logs possua uma única instância global; III. Notificar automaticamente os objetos interessados sempre que o status de um processo for alterado. Considerando os padrões de projeto GoF, apresenta a combinação correta e respectiva de padrões para atender a esses requisitos.

- a) Factory Method, Proxy e Chain of Responsibility.
- b) Factory Method, Singleton e Observer.
- c) Prototype, Bridge e Strategy.
- d) Builder, Facade e State.
- e) Abstract Factory, Adapter e Command.

FCC – 2025 – TRF - 4ª REGIÃO – Analista Judiciário - Área Apoio Especializado

3. Uma equipe de desenvolvimento está criando uma aplicação que precisa gerar diferentes tipos de relatórios (PDF, Excel ou

HTML). Cada tipo de relatório requer um processo de construção complexo e específico. Nesse cenário, o padrão de projeto criacional da Gang of Four (GoF) mais adequado para encapsular a criação de objetos complexos, permitindo a construção de diferentes representações e facilitando a adição de novos tipos de objetos sem alterar o código existente é o

- a) Builder, que separa a construção de um objeto complexo de sua representação, permitindo que o mesmo processo de construção crie diferentes representações.
- b) Prototype, que cria novos objetos copiando uma instância existente.
- c) Factory Method, que define uma interface para criar um objeto, mas permite as subclasses decidirem qual classe instanciar.
- d) Singleton, que garante que uma classe tenha apenas uma instância e fornece um ponto global de acesso a ela.
- e) Object Pool, que mantém um conjunto de objetos prontos para uso, evitando o custo de criação repetida.

FCC – 2025 – TRF - 4ª REGIÃO – Técnico Judiciário - Área Apoio Especializado

4. No desenvolvimento de aplicações para um tribunal, a equipe técnica adota padrões de projeto (GOF), boas práticas de codificação e princípios de design como SOLID, DRY e KISS. Em projetos que envolvem arquiteturas de microserviços, arquiteturas orientadas a eventos e a construção de pipelines de dados, a aplicação desses conceitos é essencial para garantir a robustez das soluções. Nesse contexto, o desenvolvimento deve seguir estratégias que

- a) agrupem funcionalidades relacionadas em módulos únicos e extensos, buscando reduzir a dispersão de código e simplificar a navegação no projeto.
- b) acelerem entrega de funcionalidades incrementais, ainda que isso implique acúmulo progressivo de dívidas técnicas no código.
- c) adotem abordagens flexíveis de implementação, dispensando o uso explícito de princípios de design, desde que os testes automatizados estejam presentes.
- d) incorporem refatoração contínua, mantenham o código alinhado aos princípios de design e favoreçam a manutenção e a evolução dos sistemas.
- e) priorizem a consistência transacional entre microserviços, ainda que isso limite o uso de

	Curso Superior em Análise e Desenvolvimento de Sistemas
	Disciplina: Padrões de Projeto, Frameworks e API
	Profª. Denise do Rocio Maciel
	LISTA DE EXERCÍCIOS: LISTA DE EXERCÍCIO DESIGN PATTERN - ETAPA 2

comunicação assíncrona em arquiteturas orientadas a eventos.

FCC – 2025 – TRT - 15ª Região (SP) – Analista Judiciário - Área Apoio Especializado - Especialidade Tecnologia da Informação

5. O Tribunal Regional do Trabalho está desenvolvendo um sistema de gestão de documentos judiciais que permite o armazenamento, recuperação e edição de diversos tipos de documentos, como petições, sentenças e relatórios de audiência. O sistema deve permitir que novas funcionalidades sejam adicionadas de forma modular e que alterações nos requisitos de cada tipo de documento não afetem o sistema como um todo. Nessas circunstâncias, a equipe de desenvolvimento deverá aplicar, corretamente, o padrão de projeto mais apropriado, que é o

- a) Singleton.
- b) Adapter.
- c) Factory Method.
- d) Composite.
- e) Decorat.

FCC – 2025 – Prefeitura de São Paulo - SP – Analista de Planejamento e Desenvolvimento Organizacional Tecnologia da Informação e Comunicação

6. A equipe de desenvolvimento de uma prefeitura está refatorando um sistema legado de atendimento ao público e precisa utilizar o padrão Factory Method para criar diferentes tipos de objetos relacionados a solicitações (como solicitações de manutenção, serviços ou emergências). A prática de implementação que reflete adequadamente o padrão Factory Method com foco em extensibilidade e encapsulamento é

- a) criar um método estático para instanciar objetos de solicitações com base em parâmetros fornecidos.
- b) delegar a criação de objetos para um método de inicialização dentro da própria classe principal.
- c) criar uma classe base que contenha a lógica para determinar o tipo de solicitação e retornar a instância apropriada.
- d) centralizar a criação de objetos em uma única classe utilitária que contém métodos estáticos para cada tipo de solicitação.
- e) utilizar classes abstratas ou interfaces para definir o método de criação e implementá-lo nas subclasses específicas.

FCC – 2025 – Prefeitura de São Paulo - SP – Analista de Planejamento e Desenvolvimento Organizacional Tecnologia da Informação e Comunicação

7. Uma prefeitura está desenvolvendo um sistema para integrar um módulo de pagamentos legados, cujo formato de dados é incompatível com o novo subsistema de cobrança online. É necessário permitir que o novo subsistema utilize o módulo antigo sem modificá-lo diretamente.

Nesse contexto, o padrão estrutural Gang of Four (GOF) que resolve de forma ideal o problema de integração entre interfaces incompatíveis é o

- a) Adapter, pois converte a interface de uma classe para outra interface esperada pelos clientes, tornando possível a comunicação entre sistemas legados e novos.
- b) Bridge, pois separa a abstração da implementação, permitindo que sistemas legados e novos se comuniquem de forma integrada.
- c) Composite, pois organiza objetos em estruturas hierárquicas, tratando composições e objetos individuais de forma integrada, permitindo a comunicação entre sistemas legados e novos.
- d) Facade, pois fornece uma interface unificada e simplificada para um conjunto complexo de subsistemas, tornando mais simples a comunicação entre sistemas legados e novos.
- e) Flyweight, pois compartilha estado intrínseco para reduzir o consumo de memória quando há múltiplas instâncias semelhantes interagindo entre sistemas ou subsistemas.

FCC – 2023 – TRT - 15ª Região (SP) – Técnico Judiciário - Área de Apoio Especializado - Especialidade Tecnologia da Informação

8. Uma Técnica de um Tribunal Regional do Trabalho utiliza padrões de projeto como soluções típicas para problemas comuns em projeto de software. Ela utilizou um deles que permite que se salve e restaure o estado anterior de um objeto sem revelar os detalhes de sua implementação. Trata-se do padrão

- a) comportamental Memento.
- b) estrutural Mediator.
- c) criacional Singleton.
- d) comportamental Visitor.
- e) estrutural Decorator.

FCC – 2005 – TRE-MG – Analista Judiciário - Análise de Sistemas

9. É um design pattern, inserido no catálogo de padrões de criação (creational) do GoF, que tem por objetivo definir uma interface para criar objetos de forma a deixar subclasses decidirem qual classe instanciar. Também conhecido como Construtor Virtual é o

- a) Prototype.
- b) Factory Method.

	Curso Superior em Análise e Desenvolvimento de Sistemas
	Disciplina: Padrões de Projeto, Frameworks e API
	Profª. Denise do Rocio Maciel
	LISTA DE EXERCÍCIOS: LISTA DE EXERCÍCIO DESIGN PATTERN - ETAPA 2

- c) Builder.
- d) Singleton.
- e) Evory Abstract.

FCC – 2021 – TJ-SC – Analista de Sistemas

10. Considerando o catálogo GoF. I. Bridge. II. Mediator. III. Proxy. Os padrões de projeto são, respectivamente, do tipo

- a) comportamental, estrutural e comportamental.
- b) criacional, estrutural e comportamental.
- c) estrutural, comportamental e criacional.
- d) estrutural, comportamental e estrutural.
- e) criacional, comportamental e criacional.

FCC – 2020 – AL-AP – Analista Legislativo - Desenvolvedor de Sistemas

11. Sobre a lista de design patterns no padrão GoF, é correto afirmar:

- a) Decorator é um padrão comportamental que categoriza um objeto para fornecer novos comportamentos.
- b) Visitor é um padrão comportamental que encapsula uma solicitação como um objeto.
- c) Abstract Factory é um padrão estrutural que permite criar novas instâncias simplesmente copiando instâncias existentes.
- d) Iterator é um padrão comportamental que fornece uma maneira de acessar sequencialmente uma coleção de objetos sem expor a sua implementação.
- e) State é um padrão criacional que centraliza operações complexas de comunicação e controle entre objetos relacionados.

FCC – 2018 – SEGEF-MA – Analista Executivo - Programador de Sistemas

12. A intenção do padrão de projeto Abstract Factory é

- a) definir uma interface para a criação de um objeto, deixando as subclasses decidirem que classe instanciar. Delega a instanciação para as subclasses.
- b) converter a interface de uma classe na interface esperada pelos clientes. Permite que classes com interfaces incompatíveis trabalhem em conjunto.
- c) fornecer uma interface para a criação de famílias de objetos relacionados ou dependentes sem especificar suas classes completas.
- d) garantir que uma classe tenha somente uma instância e fornecer um ponto global de acesso à mesma.

e) separar a construção de um objeto complexo de sua representação, de modo que o mesmo processo de construção possa criar diferentes representações.

FCC – 2019 – METRÔ-SP – Analista Desenvolvimento Gestão Júnior – Ciências da Computação

13. Considere os seguintes objetivos de padrões de projeto. 1. Separa a construção de um objeto complexo da sua representação, de forma que o mesmo processo de construção possa criar diferentes representações, ou seja, serve para encapsular a construção de um produto e permitir que ele seja construído em etapas. 2. Atribui responsabilidades adicionais a um objeto dinamicamente. Ele fornece uma alternativa flexível a subclasses para a extensão da funcionalidade, ou seja, envelopa um objeto para fornecer novos comportamentos. 3. Usa compartilhamento para dar suporte a vários objetos de forma eficiente, ou seja, permite que uma instância de uma classe possa ser usada para fornecer muitas 'instâncias virtuais'. 4. Torna possível salvar o estado de um objeto de modo que o mesmo possa ser restaurado, ou seja, permite restaurar um objeto a um dos seus estados prévios, por exemplo, quando o usuário seleciona um 'desfazer'. Correspondem, correta e respectivamente, aos objetivos de padrões de projeto:

- a) Factory Method, Decorator, Facade e Memento.
- b) Adapter, Mediator, Facade e Builder.
- c) Builder, State, Mediator e Adapter.
- d) Builder, Decorator, Flyweight e Memento.
- e) Decorator, Factory Method, Abstract Factory e State.

FCC – 2019 – TJ-MA – Analista Judiciário - Analista de Sistemas - Desenvolvimento

14. O diagrama apresenta a descrição de um design pattern GoF. Esse padrão usa as interfaces genéricas para criar os objetos concretos que fazem parte da família de objetos. O cliente não sabe ou não se importa com quais objetos concretos obtém de cada PatternConcreto, pois usa apenas as interfaces genéricas de seus produtos. Esse design pattern de criação separa detalhes da implementação de um conjunto de objetos de seu uso e é denominado (diagrama UML mostrando SoftwareCliente, interface DesignPattern com createProductA/createProductB,

	Curso Superior em Análise e Desenvolvimento de Sistemas
	Disciplina: Padrões de Projeto, Frameworks e API
	Profª. Denise do Rocio Maciel
	LISTA DE EXERCÍCIOS: LISTA DE EXERCÍCIO DESIGN PATTERN - ETAPA 2

PatternConcretoX/Y, interfaces
ProductA/ProductB e suas implementações concretas)
a) Strategy.
b) Façade.
c) Interpreter.
d) Builder.
e) Abstract Factory.

FCC – 2019 – SANASA Campinas – Analista de Tecnologia da Informação - Análise e Desenvolvimento

15. Um Analista de TI precisa criar um filtro de requisições em uma aplicação Java EE para processar um conjunto de serviços de maneira padrão. Esse recurso será usado para interceptar e manipular as requisições vindas das páginas web e as respostas retornadas da camada de modelo a estas requisições. O padrão de projeto adequado para esta situação é o
a) Intercepting Filter.
b) Filter Chain.
c) Bridge.
d) interpreter.
e) Filter Controller

FCC – 2019 – TRF - 4ª REGIÃO – Analista Judiciário - Sistemas de Tecnologia da Informação

16. Uma Analista de TI quer criar um aplicativo capaz de trocar mensagens entre diversas plataformas móveis, um Android enviando mensagem para um iOS, um Windows Phone trocando mensagens com um Android etc. O problema é que cada uma destas plataformas implementa maneiras diferentes de receber mensagens. Ela sabe que criar vários métodos para cada plataforma não seria uma solução adequada, pois, se o aplicativo precisar incluir uma nova plataforma, outros métodos teriam que ser criados. A modelagem de relacionamentos muitos para muitos pode deixar o projeto do sistema demasiadamente complexo. A Analista procurou, então, um padrão de projeto para utilizar na solução. Dentre os padrões GoF, encontrou um que permite definir um objeto que encapsula a forma como um conjunto de objetos interage. Este padrão promove o acoplamento fraco ao evitar que os objetos se refiram uns aos outros explicitamente e permite variar o uso da interação de forma independente. Atua como um interlocutor entre relacionamentos muitos para muitos, ao evitar uma referência explícita aos objetos, sendo adequado ao problema. Trata-se do padrão comportamental

a) Mediator.
b) Composite.
c) Adapter.
d) Factory Method.
e) Prototype.

FCC – 2016 – CREMESP – Analista de Tecnologia da Informação - Análise de Sistemas

17. Suponha que exista uma interface ITraduzFrase que define o método imprimeFrase(). Existem, também, as classes InglesTraduzFrase, EspanholTraduzFrase e AlemaoTraduzFrase que estendem a interface ITraduzFrase e utilizam o método imprimeFrase(). Um Analista utilizou um design pattern que cria objetos de forma dinâmica sem conhecer a classe de implementação. A classe FrasePadrao implementa esse padrão de forma que, de acordo com o idioma desejado, retorna um objeto do tipo ITraduzFrase. O objetivo é que, sendo fornecido o idioma, a frase será nele impressa, em inglês, espanhol ou alemão. Considerando o exemplo e sabendo que a definição do padrão é 'Define uma interface para criar um objeto, mas permite às classes decidirem qual classe instanciar. Este padrão permite a uma classe deferir a instanciación para subclasses.', o nome do padrão usado pelo Analista é
a) Strategy.
b) Factory Method.
c) Singleton.
d) Chain of Responsibility.
e) Adapter.

FCC – 2017 – TRF - 5ª REGIÃO – Analista Judiciário - Informática Desenvolvimento

18. O padrão de projeto comportamental que define uma dependência de um-para-muitos entre objetos de modo que, quando um objeto muda de estado, todos os seus dependentes são notificados e atualizados automaticamente é o
a) Facade.
b) Observer.
c) Adapter.
d) Abstract Factory.
e) Prototype.

FCC – 2013 – AL-RN – Técnico Legislativo - Programador

19. Em aplicações para web orientadas a objeto, há um design pattern muito utilizado, (relacionado à arquitetura de sistemas) que possui um componente que: – Define o comportamento da aplicação. – Mapeia ações do

	Curso Superior em Análise e Desenvolvimento de Sistemas
	Disciplina: Padrões de Projeto, Frameworks e API
	Profª. Denise do Rocio Maciel
	LISTA DE EXERCÍCIOS: LISTA DE EXERCÍCIO DESIGN PATTERN - ETAPA 2

usuário para classes que cuidam do acesso aos dados. – Selecciona páginas web para entregar resultados vindos das classes de acesso aos dados. O componente citado no texto acima é o

- a) Business.
- b) Controller.
- c) Persistence.
- d) Middle.
- e) Core.

FCC – 2017 – DPE-RS – Analista - Desenvolvimento de Sistemas 20.

Considere que é necessário fazer um programa que vá funcionar em várias plataformas, como Windows, Linux, Mac OS etc. O programa fará uso de diversas abstrações de janelas gráficas, por exemplo, janela de diálogo, janela de aviso, janela de erro etc. Um Analista sugeriu o uso de um padrão de projeto GoF estrutural que fornece um nível de abstração maior que o Adapter, pois são separadas as implementações e as abstrações, permitindo que cada uma varie independentemente. O padrão sugerido pelo Analista é o

- a) Bridge e as implementações seriam as classes de janela das plataformas.
- b) Prototype, pois especifica os tipos de plataformas a serem criadas usando uma instância protótipo e cria novas interfaces pela cópia desse protótipo.
- c) MVC, de forma que a camada Model fique responsável por acionar as janelas, a camada Controller exiba os dados do Model ao usuário e a camada View armazene as classes que fazem a comunicação com o Banco de Dados.
- d) Visitor e as interfaces seriam executadas nos elementos de cada plataforma.
- e) State, que permite a uma plataforma alterar o comportamento de sua interface.

FCC – 2017 – ARTESP – Agente de Fiscalização à Regulação de Transporte - Tecnologia de Informação

21. Suponha que em uma organização exista um conjunto de cargos e para cada cargo existem regras de cálculo da taxa de produtividade, cujo valor é determinado por uma porcentagem do salário base do funcionário. Este valor é acrescido ao salário do funcionário como um bônus. Um desenvolvedor deve criar uma solução que utilize um design pattern (padrão de projeto) cuja intenção é: 'definir uma família de algoritmos, encapsular cada uma delas e torná-las intercambiáveis'. Ou seja, o padrão sugere que algoritmos parecidos (métodos de cálculo da taxa

de produtividade) sejam separados de quem os utiliza (funcionário). O padrão, além de encapsular os algoritmos da mesma família, também permite a reutilização do código. Nesta situação, caso a regra para cálculo da taxa de produtividade de dois tipos de funcionários, como Desenvolvedor e DBA, seja a mesma, não é necessário escrever código extra. Caso seja necessário incluir um novo cargo, basta implementar sua estratégia de cálculo da taxa ou reutilizar outra similar. Nenhuma outra parte do código precisa ser alterada. O design pattern mencionado na situação apresentada é

- a) Abstract Method.
- b) Builder.
- c) Prototype.
- d) Strategy.
- e) Singleton.

FCC – 2017 – TRE-SP – Técnico Judiciário – Programação de Sistemas

22. Um Programador de Sistemas teve que desenvolver algumas aplicações utilizando padrões de projeto. Sua primeira necessidade foi utilizar um padrão para separar a construção de objetos complexos da sua representação, de forma que o mesmo processo de construção pudesse criar diferentes representações. A segunda necessidade foi o uso de um padrão para permitir que classes incompatíveis trabalhassem em conjunto, já que converte a interface de uma classe em outra, para atender às expectativas do cliente. Para atender suas necessidades, o Programador usou, correta e respectivamente, os padrões de projeto

- a) Factory Method e Singleton.
- b) Builder e Abstract Factory.
- c) Builder e Adapter.
- d) Singleton e Abstract Factory.
- e) Adapter e Bridge.

FCC – 2016 – TRT - 20ª REGIÃO (SE) – Técnico Judiciário - Tecnologia da Informação

23. Um desenvolvedor precisa utilizar um padrão de projeto para interceptar e manipular requisições HTTP de entrada de usuários ao sistema web, e respostas de saída através de filtros de pré-processamento e pós-processamento. Além disso, precisa utilizar outro padrão de projeto capaz de separar as regras de negócio da aplicação das regras de acesso ao banco de dados, permitindo assim centralizar em classes específicas, as operações de conexão ao

	Curso Superior em Análise e Desenvolvimento de Sistemas
	Disciplina: Padrões de Projeto, Frameworks e API
	Profª. Denise do Rocio Maciel
	LISTA DE EXERCÍCIOS: LISTA DE EXERCÍCIO DESIGN PATTERN - ETAPA 2

banco de dados e realização de operações SQL. Os padrões de projeto que o desenvolvedor terá que utilizar são

- a) Front Controller e Data Transfer Object.
- b) Application Filter e Data Session Façade
- c) Intercepting Filter e Data Access Object.
- d) Controller Helper e Data Manager Object.
- e) Application Controller e Data Persistent Object.

FCC – 2016 – Prefeitura de Teresina - PI – Técnico de Nível Superior - Analista de Sistemas

24. Dentre os tipos de Padrões de Projeto (Design Patterns) o que se caracteriza por definir uma interface para a criação de um objeto e que permite que a subclasse decida qual classe instanciar é denominado

- a) Factory Method.
- b) Builder.
- c) Prototype.
- d) Abstract Factory.
- e) Composite.

FCC – 2016 – TRT - 14ª Região (RO e AC) – Analista Judiciário - Tecnologia da Informação

25. Um Analista precisa construir uma aplicação que possui uma função da qual se conhece somente o algoritmo de execução, sendo que o trabalho de codificação para realização da operação desta função tem que ser postergado. Ele optou pelo uso do padrão de projeto comportamental

- a) Template Method, que permite implementar as partes invariantes de um algoritmo e deixar para subclasses a implementação da parte variante.
- b) Adapter, que cria classes reutilizáveis que cooperam com classes não relacionadas ou não previstas.
- c) Singleton, que possibilita o uso de uma instância estendida sem alterar o seu código.
- d) Abstract Factory, que permite criar uma biblioteca de classes de produtos e revelar somente suas interfaces, não suas implementações.
- e) Factory Method, que define uma interface para criar objetos, mas deixa que as subclasses decidam que classe instanciar.

FCC – 2016 – TRT - 14ª Região (RO e AC) – Técnico Judiciário - Tecnologia da Informação

26. Os padrões de projeto

- a) podem deixar um sistema mais complexo ou degradar a sua performance. O seu uso indevido ou inadequado para um determinado contexto constitui-se em um anti pattern.

b) sempre criam flexibilidade e variabilidade pela introdução de níveis adicionais de endereçamento indireto. Como melhoram o desempenho do sistema devem ser sempre aplicados.

c) comportamentais abstraem ou adiam o processo de criação dos objetos, ajudando a tornar o sistema dependente de como seus objetos são criados, compostos e representados.

d) estruturais se concentram nos algoritmos de herança entre os objetos. Eles não descrevem apenas padrões de objetos ou de classes, mas também os padrões de comunicação entre os objetos.

e) de criação se preocupam com a forma como classes e objetos são compostos para formar estruturas maiores. Utilizam o polimorfismo para compor interfaces ou implementações.

FCC – 2016 – TRT - 23ª REGIÃO (MT) – Analista Judiciário - Tecnologia da Informação

27. Um Analista de Sistemas necessita utilizar alguns padrões de projeto para solucionar determinados problemas. De acordo com as Categorias de Padrões do Gang of Four– GoF, ele vai utilizar um padrão de cada uma das três categorias, pela ordem, a saber: – Padrões de Criação – Padrões Estruturais – Padrões Comportamentais. Em relação a essas três categorias, os padrões são, respectivamente,

- a) Strategy, Template Method e Decorator.
- b) Factory Method, Flyweight e Memento.
- c) Bridge, Prototype e Composite.
- d) Abstract Factory, Singleton e Observer
- e) Façade, Proxy e Iterator.

FCC – 2016 – TRT - 23ª REGIÃO (MT) – Técnico Judiciário - Tecnologia da Informação

28. Em muitas situações uma 'aplicação cliente' precisa criar objetos cuja construção efetiva só é definida em tempo de execução. A 'aplicação cliente' não deve se preocupar com a criação dos objetos. Trata-se da motivação do padrão de projeto

- a) Abstract Factory.
- b) Builder.
- c) Factory Method.
- d) Singleton.
- e) Proxy.

FCC – 2016 – TRT - 23ª REGIÃO (MT) – Técnico Judiciário - Tecnologia da Informação

29. Usar compartilhamento para suportar eficientemente grandes quantidades de objeto de

	Curso Superior em Análise e Desenvolvimento de Sistemas
	Disciplina: Padrões de Projeto, Frameworks e API
	Profª. Denise do Rocio Maciel
	LISTA DE EXERCÍCIOS: LISTA DE EXERCÍCIO DESIGN PATTERN - ETAPA 2

granularidade fina é intenção do padrão de projeto

- a) Command.
- b) Composite.
- c) Prototype.
- d) Interpreter.
- e) Flyweight.

FCC – 2015 – DPE-SP – Programador

30. Um Programador está desenvolvendo uma aplicação em que os objetos mudam de estado com muita frequência. Em tempo de execução, a mudança no estado lógico destes objetos implica também na alteração em seu comportamento. Nessa aplicação é ideal que o Programador utilize um design pattern comportamental cuja classe Context é a interface principal para as requisições dos clientes. O Programador deve utilizar o design pattern

- a) Prototype
- b) State.
- c) Adapter.
- d) Singleton.
- e) Façade.

FCC – 2015 – DPE-RR – Analista de Sistemas

31. Um Analista de Sistemas, em função da extrema tensão causada pelo curto prazo de entrega do novo software, concentrou a maior parte do sistema em uma classe central com dezenas de métodos e atributos. Os métodos foram obtidos de soluções prontas disponíveis nas comunidades de desenvolvedores e ele conseguiu entregar o sistema funcionando no prazo. O Analista

- a) utilizou anti-patterns consagrados para os métodos.
- b) criou a classe central baseado no design pattern Singleton.
- c) utilizou herança ao criar os métodos.
- d) aplicou os princípios do design pattern conhecido como Input Kludge
- e) incorreu em uma solução anti-pattern conhecida como the blob.

FCC – 2015 – TRE-PB – Técnico Judiciário - Área Apoio Especializado - Programação de Sistemas

32. Um técnico deseja usar um padrão de projeto de criação que permita que as subclasses da aplicação possam variar. Este padrão deverá ser focado no processo de instanciação e encapsular a criação de objetos, deixando as subclasses decidirem quais objetos criar e

garantindo assim, baixo acoplamento. Para conseguir o que deseja, o técnico selecionou o padrão de projeto que possui uma classe abstrata Creator que define um método específico para criação de objetos. Trata-se do padrão:

- a) Prototype.
- b) Adapter.
- c) Factory Method.
- d) Composite.
- e) Façade.

FCC – 2015 – DPE-SP – Analista de Sistemas

33. A empresa em que um Analista trabalha se comprometeu com um cliente em satisfazer todos os requisitos para uma determinada aplicação de software no prazo previsto, porém, de fato, não vai conseguir cumprir o prometido, mesmo assim, vem apresentando funções não implementadas como se já tivessem sido construídas. Percebe-se, nesse caso, o antipattern de gerenciamento de projetos conhecido como

- a) Chain of Responsibility.
- b) Fire Drill.
- c) Abstract Factory.
- d) Smoke and Mirrors.
- e) Analysis Paralysis.

FCC – 2015 – MPE-PB – Analista de Sistemas – Desenvolvedor

34. Considere a seguinte situação: Uma grande empresa possui uma rede de representação de seus produtos constituída por escritórios de venda em diversas cidades. Cada escritório possui um conjunto de vendedores que vão até os varejistas oferecer os produtos. Varejistas podem estar em áreas populosas, como supermercados, ou em áreas remotas, como comércios de bairros rurais. Ao chegar no varejista, o vendedor utiliza um sistema fornecido pela empresa para registrar os pedidos de produtos. Na situação descrita, o objetivo é registrar os pedidos na base de dados da central de distribuição da empresa. Porém, por problemas de comunicação com a central, o computador portátil do varejista pode não conseguir conectar com a base central para registrar os pedidos. Se isso ocorrer, o sistema deve tentar salvá-los na base de dados do escritório. Quando a base do escritório também não estiver acessível, os pedidos deverão ser salvos na base de dados local, no próprio computador portátil do vendedor. Para estruturar a solução para o problema da empresa, o

	Curso Superior em Análise e Desenvolvimento de Sistemas
	Disciplina: Padrões de Projeto, Frameworks e API
	Profª. Denise do Rocio Maciel
	LISTA DE EXERCÍCIOS: LISTA DE EXERCÍCIO DESIGN PATTERN - ETAPA 2

Analista de Sistemas utilizou um padrão de projeto que tem a intenção de evitar o acoplamento do remetente de uma solicitação ao seu receptor, ao dar a mais de um objeto a oportunidade de tratar a solicitação e encadear os objetos receptores, passando a solicitação ao longo da cadeia até que um objeto a trate. O padrão de projeto utilizado é um dos padrões comportamentais da GoF (Gang of Four), denominado

- a) Adapter.
- b) Composite Method.
- c) Chain of Responsibility.
- d) Factor Method.
- e) Prototype.

FCC – 2015 – TRT - 3ª Região (MG) – Técnico Judiciário - Área Tecnologia da Informação

35. Considere: I. A aplicação cliente interage com uma fábrica genérica de objetos. II. Os objetos serão gerados efetivamente pela fábrica concreta que estiver sendo utilizada no momento. III. A aplicação cliente não precisa ser configurada para interagir com cada uma das fábricas concretas. IV. Novas fábricas concretas podem ser agregadas, alteradas ou retiradas do sistema sem necessidade de alterações na aplicação cliente. Estas são características e propriedades do padrão de projeto

- a) Factory Method.
- b) Builder
- c) Prototype.
- d) Abstract Factory.
- e) Singleton.

FCC – 2015 – TRT - 15ª Região (SP) – Analista Judiciário - Tecnologia da Informação

36. Os padrões de projeto tornam mais fácil reutilizar projetos e arquiteturas bem sucedidas. Atualmente existem diversos padrões de projetos conforme abaixo: I. Fornece uma interface para a criação de famílias de objetos relacionados ou dependentes sem especificar suas classes concretas. II. Converte a interface de uma classe em outra interface esperada pelos clientes permitindo que certas classes trabalhem em conjunto, pois de outra forma, seria impossível por causa de suas interfaces incompatíveis. III. Fornece uma maneira de acessar sequencialmente os elementos de uma agregação de objetos sem expor sua representação subjacente. Os padrões de projeto apresentados em I, II e III são, respectivamente,

- a) Façade, Builder e Mediator.
- b) Abstract Factory, Adapter e Iterator.
- c) Façade, Adapter e Interpreter.
- d) Singleton, Builder e Mediator.
- e) Abstract Factory, Prototype e Iterator.

FCC – 2015 – CNMP – Analista do CNMP - Tecnologia de Informação e Comunicação Desenvolvimento Sistemas

37. Um Analista de Desenvolvimento de Sistemas do CNMP deve indicar o padrão de projeto mais adequado para ser aplicado na seguinte situação: Uma aplicação que existe simultaneamente em um dispositivo móvel e no ambiente corporativo, necessita de um processo de sincronização entre as informações processadas no dispositivo móvel e na base corporativa. Ambas as aplicações devem se comunicar com um objeto que deve ser único para processar este sincronismo, a fim de evitar a possibilidade de criar dados na base. O padrão de projeto corretamente indicado pelo Analista deve ser

- a) Factory Method, um padrão de criação, que busca definir o fluxo de um algoritmo em uma operação, postergando (deferring) alguns passos para subclasses, sem mudar a estrutura do mesmo.
- b) Prototype, um padrão estrutural, que busca fornecer uma interface para criação de famílias de objetos relacionados ou dependentes sem especificar suas classes concretas.
- c) Singleton, um padrão de criação, que busca garantir que um objeto terá apenas uma única instância, ou seja, uma classe irá gerar apenas um objeto e que este estará disponível de forma única para todo o escopo de uma aplicação.
- d) Command, um padrão comportamental, que busca definir o fluxo de um algoritmo em uma operação, postergando (deferring) alguns passos para subclasses, sem mudar a estrutura do mesmo.
- e) Façade, um padrão estrutural, que busca garantir que um objeto terá apenas uma única instância, ou seja, uma classe irá gerar apenas um objeto e que este estará disponível de forma única para todo o escopo de uma aplicação.

FCC – 2014 – TRT - 13ª Região (PB) – Analista Judiciário - Tecnologia da Informação

38. Angela pretende utilizar alguns design patterns em seu projeto Java e, após algumas pesquisas, encontrou o que buscava em Singleton e Prototype cujos objetivos são,

	Curso Superior em Análise e Desenvolvimento de Sistemas
	Disciplina: Padrões de Projeto, Frameworks e API
	Profª. Denise do Rocio Maciel
	LISTA DE EXERCÍCIOS: LISTA DE EXERCÍCIO DESIGN PATTERN - ETAPA 2

respectivamente: I. Encapsular a escolha das classes concretas a serem utilizadas na criação dos objetos de diversas famílias. II. Permitir a criação de uma única instância de uma classe e fornecer um modo para recuperá-la. III. Possibilitar o reaproveitamento de objetos. IV. Possibilitar a criação de novos objetos a partir da cópia de objetos existentes. Está correto o que consta APENAS em

- a) I e II.
- b) I e III.
- c) II e III.
- d) II e IV
- e) III e IV.

FCC – 2014 – TRT - 13ª Região (PB) – Técnico Judiciário - Tecnologia da Informação

39. Os padrões de projeto estruturais diminuem o acoplamento entre os objetos de um sistema orientado a objetos tornando mais simples eventuais alterações no funcionamento do sistema. Um destes padrões permite centralizar todas as requisições a uma aplicação Web, ou seja, haverá um único ponto de entrada na aplicação que receberá e tratará todas as requisições. Este padrão é conhecido como

- a) Bridge.
- b) Request Controller.
- c) Manager Request.
- d) Front Controller.
- e) Facade.

FCC – 2013 – TRT - 5ª Região (BA) – Técnico Judiciário - Tecnologia da Informação

40. Design patterns podem ser divididos em três categorias: creational, structural e behavioral. Um dos design patterns da categoria creational garante para uma classe específica, apenas uma única instância, a qual é acessível de forma global e uniforme. Este design pattern é conhecido como

- a) Façade.
- b) Builder.
- c) Abstract Factory.
- d) Singleton.
- e) Composite.

FCC – 2014 – TRT - 16ª REGIÃO (MA) – Técnico Judiciário - Tecnologia da Informação

41. Em relação aos padrões de projeto é INCORRETO afirmar:

- a) São uma porção identificada de conhecimento profundo, que transmite a essência de uma

solução provada para um problema recorrente em um certo contexto.

b) Descrevem uma estrutura de projeto que resolve um problema particular em um cenário específico e podem ter impacto na maneira pela qual o padrão é aplicado e usado.

c) Cada padrão de projeto descreve um problema que ocorre repetidas vezes em um ambiente e, então, descreve o núcleo da solução daquele problema, de tal modo que se pode usar esta solução diversas vezes.

d) A descrição de um padrão de projeto pode considerar um conjunto de influências de projeto que descrevem o ambiente e as condições que devem existir para torná-lo aplicável.

e) Padrões arquiteturais e padrões de código são exemplos de padrões de projeto. Os arquiteturais são padrões específicos de uma plataforma e possuem padrões específicos de código, como o Java.

FCC – 2013 – TRT - 15ª Região (SP) – FCC - 2013 - TRT - 15ª Região - Analista Judiciário - Tecnologia da Informação

42. Os padrões Gang of Four (GoF) organizam um conjunto de padrões de projeto (design patterns) em três grupos: de criação, estruturais e comportamentais. Três destes padrões são descritos a seguir: I. Em situações em que classes precisam trabalhar juntas, mas isto não está sendo possível porque suas interfaces são incompatíveis, pode-se utilizar este design pattern que permite converter a interface de uma classe em outra interface esperada pelos clientes de forma que classes com interfaces incompatíveis possam interagir. II. Este design pattern pode ser utilizado quando se deseja definir uma dependência um-para-muitos entre objetos de modo que quando um objeto muda o estado, todos seus dependentes são notificados e atualizados. III. Em situações em que se deseja acessar o conteúdo de uma coleção sem expor sua representação interna utiliza-se este design pattern que permite prover uma interface única para varrer coleções diferentes. Os padrões descritos nos itens I, II e III são, respectivamente,

- a) Adapter, Facade e Strategy.
- b) Prototype, Composite e Command.
- c) Abstract Factory, Observer e Iterator.
- d) Adapter, Observer e Iterator.
- e) Abstract Factory, Composite e Command.

FCC – 2013 – AL-RN – Analista Legislativo - Analista de Sistemas

	Curso Superior em Análise e Desenvolvimento de Sistemas
	Disciplina: Padrões de Projeto, Frameworks e API
	Profª. Denise do Rocio Maciel
	LISTA DE EXERCÍCIOS: LISTA DE EXERCÍCIO DESIGN PATTERN - ETAPA 2

43. Analise as seguintes afirmações: I. Fornece uma interface para a criação de uma família de objetos relacionados ou dependentes sem fornecer os detalhes de implementação das classes concretas. II. Converte uma interface de uma classe existente em outra interface esperada pelos clientes. Permite que algumas classes com interfaces diferentes trabalhem em conjunto. III. Separa uma implementação de sua abstração, de forma que ambas possam variar independentemente. IV. Separa a construção de um objeto complexo de sua representação, de modo que o mesmo processo possa criar representações diferentes. Tratam, respectivamente, dos design patterns:

- a) Builder - Adapter - Bridge - Abstract Factory.
- b) Abstract Factory - Adapter - Bridge - Builder.
- c) Bridge - Adapter - Builder - Abstract Factory.
- d) Adapter - Builder - Abstract Factory - Bridge.
- e) Builder - Bridge - Abstract Factory - Adapter.

FCC – 2013 – DPE-SP – Programador de computador

44. Um design pattern descreve uma solução geral comprovada e reutilizável para um problema recorrente no desenvolvimento de sistemas de software orientados a objetos. Padrões de projeto ajudam a reconhecer e implementar boas soluções para problemas comuns. Dois dos principais design patterns utilizados atualmente são descritos a seguir: I. Visa garantir que uma classe só tenha uma única instância e prover um ponto de acesso global a ela. II. Visa definir uma dependência um-para-muitos entre objetos para que quando um objeto mudar de estado os seus dependentes sejam notificados e atualizados automaticamente. Os design patterns descritos em I e II são, respectivamente:

- a) Singleton e Observer.
- b) Facade e Adapter.
- c) Composite e Adapter.
- d) Singleton e Command.
- e) Facade e Observer.

FCC – 2011 – TCE-PR – Analista de Controle - Informática

45. Os design patterns

- a) são projetos de arquitetura para um domínio específico de aplicação e sempre trazem componentes predefinidos que envolvem código de programação.
- b) consistem em conjuntos de classes que um usuário instancia para utilizar seus métodos. Após a chamada ao método, o controle do fluxo da aplicação retorna para o usuário.

- c) são de uso exclusivo em processos de desenvolvimento de soluções orientadas a objetos, já que os objetos são a mais adequada abstração para o reúso.
- d) são aplicações propriamente ditas, normalmente construídas pela integração de diversos frameworks.
- e) podem ser modelados utilizando-se a linguagem UML que fornece um meio eficiente de modelar padrões de projeto representando-os como colaborações.

FCC – 2011 – TRE-RN – Programador de computador

46. Na engenharia de software, os padrões de projetos comportamentais tratam das interações e divisões de responsabilidades entre as classes ou objetos. São exemplos típicos dessa família:

- a) Command, Factory Method e Prototype.
- b) Builder, Prototype e Singleton.
- c) Chain of Responsibility, Interpreter e Iterator.
- d) Adapter, Bridge e Façade.
- e) Abstract Factory, Builder e Composite.

FCC – 2011 – TRT - 14ª Região (RO e AC) – Analista Judiciário - Tecnologia da Informação

47. No contexto dos padrões de projeto: I. Oferecer uma interface simples para uma coleção de classes. II. Desacoplar uma abstração de sua implementação para que ambas possam variar independentemente. Correspondem respectivamente a

- a) Façade e Bridge.
- b) Adapter e Façade.
- c) Composite e Bridge.
- d) Façade e Composite.
- e) Bridge e Adapter.

FCC – 2011 – TRT - 4ª REGIÃO (RS) – Analista Judiciário - Tecnologia da Informação

48. O catálogo de padrões de projeto (design patterns) do GoF contém

- a) 20 padrões e está basicamente dividido em duas seções: Structural e Behavioral.
- b) 21 padrões e está basicamente dividido em duas seções: Creational e Behavioral.
- c) 23 padrões e está basicamente dividido em duas seções: Structural e Behavioral.
- d) 23 padrões e está, basicamente, dividido em três seções: Creational, Structural e Behavioral.
- e) 24 padrões e está basicamente dividido em três seções: Creational, Spectral e Behavioral.

	Curso Superior em Análise e Desenvolvimento de Sistemas
	Disciplina: Padrões de Projeto, Frameworks e API
	Profª. Denise do Rocio Maciel
	LISTA DE EXERCÍCIOS: LISTA DE EXERCÍCIO DESIGN PATTERN - ETAPA 2

FCC – 2011 – TRT - 24ª REGIÃO (MS) – Analista Judiciário - Tecnologia da Informação

49. Considere: I. Fornecer uma interface para criação de famílias de objetos relacionados ou dependentes, sem especificar suas classes concretas. Possibilitar o adiamento da instanciación para as subclasses. II. Garantir a existência de apenas uma instância de uma classe, mantendo um ponto global de acesso ao seu objeto. III. Possibilitar o armazenamento do estado interno de um objeto em um determinado momento, para que seja possível retorná-lo a esse estado, caso necessário. I, II e III são, respectivamente, objetivos dos design patterns intitulados:

- a) Interpreter, Iterator e Memento.
- b) Command, Singleton e Iterator.
- c) Factory Method, Singleton e Memento.
- d) Iterator, Factory Method e Flyweight.
- e) Singleton, Flyweight e Command.

FCC – 2010 – TRT - 22ª Região (PI) – Analista Judiciário - Tecnologia da Informação

50. Creational Pattern Java que tem por objetivo separar a construção de um objeto complexo da sua representação, de tal forma que diversas representações podem ser criadas em razão da necessidade do programa. Trata-se de

- a) Flyweight pattern.
- b) Builder pattern.
- c) Prototype pattern.
- d) Singleton pattern.
- e) Bridge pattern.

FCC – 2010 – TRT - 9ª REGIÃO (PR) – Analista Judiciário - Tecnologia da Informação

51. Sobre design pattern considere: I. No framework pode incluir código de programação e conter vários design patterns. II. No design pattern pode incluir código de programação e conter vários frameworks. III. Os design patterns são bastante abstratos e os frameworks menos abstratos. Está correto o que consta em

- a) I e III, apenas.
- b) I e II, apenas.
- c) II e III, apenas.
- d) III, apenas.
- e) I, II e III.

FCC – 2009 – TJ-PI – Analista Judiciário - Tecnologia da Informação

52. Acerca desses padrões, avalie: I. Em termos de padrões de criação de projetos, algumas classes, tais como um spooler de

impressão ou um sistema de arquivos, devem ser instanciadas uma única vez e prover um ponto de acesso global a esta instância. II. Pertencente ao grupo de padrões estruturais, é utilizado para permitir que dois objetos se comuniquem mesmo que tenham interfaces incompatíveis. III. Encapsula uma requisição como um objeto, permitindo que os clientes parametrizem diferentes requisições e filas, ou façam o registro de log de requisições e dêem suporte a operações que podem ser desfeitas. IV. Atribuir a responsabilidade do tratamento de um evento do sistema a uma classe que representa o próprio sistema ou à classe que representa um caso de uso. As proposições correspondem, respectivamente, aos padrões

- a) Singleton, Adapter, Command e Controller.
- b) Command, Iterator, Singleton e Expert.
- c) Façade, Iterator, Expert e Adapter.
- d) Controller, Expert, Singleton e Prototype.
- e) Adapter, Façade, Command e Iterator.

FCC – 2009 – TJ-PI – Analista Judiciário - Tecnologia da Informação

53. Os padrões de projetos, quando aplicados ao desenvolvimento de aplicações, fornecem meios de descrever soluções comuns para problemas comuns, resultando em redução de tempo gasto com o desenvolvimento e melhoria da qualidade da aplicação. Analise: I. É o responsável pela especificação dos tipos de objetos a serem criados usando uma 'instância' prototípica e pela criação de novos objetos copiando este protótipo. II. Define uma interface de nível mais alto que torna o subsistema mais fácil de usar e fornece uma interface única para um subsistema com diversas interfaces; compõe o grupo de padrões estruturais. III. Integrante do grupo de padrões comportamentais, ele provê uma forma de acessar sequencialmente os elementos de um agregado de objetos, sem expor a representação interna desse agregado. IV. As consequências do uso deste padrão é que o encapsulamento é mantido, já que objetos usam sua própria informação para cumprir responsabilidades; leva ao fraco acoplamento entre objetos e à alta coesão, uma vez que objetos fazem tudo que é relacionado à sua própria informação. As afirmações correspondem, respectivamente, aos padrões

- a) Command, Iterator, Singleton e Expert.
- b) Controller, Expert, Singleton e Prototype.
- c) Command, Singleton, Controller e Façade.

	Curso Superior em Análise e Desenvolvimento de Sistemas
	Disciplina: Padrões de Projeto, Frameworks e API
	Profª. Denise do Rocio Maciel
	LISTA DE EXERCÍCIOS: LISTA DE EXERCÍCIO DESIGN PATTERN - ETAPA 2

- d) Prototype, Façade, Iterator e Expert.
e) Adapter, Façade, Command e Iterator.

FCC – 2010 – TRF - 4ª REGIÃO – Analista Judiciário - Tecnologia da Informação

54. Sobre os design patterns, é correto afirmar:

- a) Padrões e linguagens de padrões são maneiras de implementar sistemas orientados a objetos por meio da captação da experiência de programadores. Os padrões, apesar de abstratos, sempre incluem algum código de programação.
b) São aplicações, propriamente ditas, dedicadas aos domínios de aplicações específicos, tais como sistemas de telecomunicações ou financeiros.
c) Não são complexos e necessita-se de um tempo mínimo para aprender a usá-los.
d) O princípio geral de englobamento de experiência em um padrão é aplicável apenas à abordagem de projeto de software orientado a objetos.
e) O padrão é uma descrição de conhecimento e experiência acumulados, uma solução comprovada para um problema comum.

FCC – 2010 – TRE-AM – Analista Judiciário - Tecnologia da Informação

55. Em relação aos padrões de projetos orientados a objetos, separar dados ou lógica de negócios da interface do usuário e do fluxo da aplicação para permitir que uma mesma lógica de negócios possa ser acessada e visualizada através de várias interfaces, sem saber de quantas nem quais interfaces com o usuário estão exibindo seu estado, é característica do padrão de projeto arquitetural

- a) Observer.
b) Layers.
c) Model-View-Controller.
d) Expert.
e) Creator.

FCC – 2026 – TJ-CE – Analista Judiciário - Área Técnico Administrativa

56. Uma equipe de arquitetura de software de um Tribunal de Justiça iniciou a modernização incremental de um sistema processual monolítico. A estratégia prevê que determinadas capacidades do sistema legado sejam substituídas por novos serviços, sem interrupção global da operação. Considerando o padrão Strangler Fig, a conduta tecnicamente coerente é

- a) concluir os serviços planejados antes da troca operacional, reduzindo o período de convivência entre as arquiteturas antiga e nova.
b) duplicar as funções do legado em serviços equivalentes e permitir que cada unidade usuária defina qual versão utilizará.
c) iniciar a troca por bibliotecas e componentes técnicos comuns, mantendo as regras de negócio no monólito até a etapa final.
d) selecionar uma capacidade do legado, implementar sua nova versão em serviço separado e encaminhar gradualmente o tráfego correspondente para essa implementação.
e) preservar as funções existentes no monólito e usar os novos serviços para requisitos ainda não atendidos pela aplicação original.

FCC – 2026 – MPE-AL – Analista do Ministério Público

57. Para desenvolver um módulo de notificações processuais conforme o perfil do réu, o sistema deve atender aos seguintes requisitos: I. Instanciar diferentes tipos de alertas (e-mail, SMS, Push), permitindo a inclusão de novos tipos sem alteração do código existente; II. Garantir que o registrador de logs possua uma única instância global; III. Notificar automaticamente os objetos interessados sempre que o status de um processo for alterado. Considerando os padrões de projeto GoF, apresenta a combinação correta e respectiva de padrões para atender a esses requisitos.

- a) Factory Method, Proxy e Chain of Responsibility.
b) Factory Method, Singleton e Observer.
c) Prototype, Bridge e Strategy.
d) Builder, Facade e State.
e) Abstract Factory, Adapter e Command.

FCC – 2025 – TRF - 4ª REGIÃO – Analista Judiciário - Área Apoio Especializado

58. Uma equipe de desenvolvimento está criando uma aplicação que precisa gerar diferentes tipos de relatórios (PDF, Excel ou HTML). Cada tipo de relatório requer um processo de construção complexo e específico. Nesse cenário, o padrão de projeto criacional da Gang of Four (GoF) mais adequado para encapsular a criação de objetos complexos, permitindo a construção de diferentes representações e facilitando a adição de novos tipos de objetos sem alterar o código existente é o

a) Builder, que separa a construção de um objeto complexo de sua representação, permitindo que o

	Curso Superior em Análise e Desenvolvimento de Sistemas
	Disciplina: Padrões de Projeto, Frameworks e API
	Profª. Denise do Rocio Maciel
	LISTA DE EXERCÍCIOS: LISTA DE EXERCÍCIO DESIGN PATTERN - ETAPA 2

mesmo processo de construção crie diferentes representações.

- b) Prototype, que cria novos objetos copiando uma instancia existente.
- c) Factory Method, que define uma interface para criar um objeto, mas permite as subclasses decidirem qual classe instanciar.
- d) Singleton, que garante que uma classe tenha apenas uma instancia e fornece um ponto global de acesso a ela.
- e) Object Pool, que mantém um conjunto de objetos prontos para uso, evitando o custo de criação repetida.

FCC – 2025 – TRF - 4ª REGIÃO – Técnico Judiciário - Área Apoio Especializado

59. No desenvolvimento de aplicações para um tribunal, a equipe técnica adota padrões de projeto (GOF), boas práticas de codificação e princípios de design como SOLID, DRY e KISS. Em projetos que envolvem arquiteturas de microsserviços, arquiteturas orientadas a eventos e a construção de pipelines de dados, a aplicação desses conceitos é essencial para garantir a robustez das soluções. Nesse contexto, o desenvolvimento deve seguir estratégias que
- a) agrupem funcionalidades relacionadas em módulos únicos e extensos, buscando reduzir a dispersão de código e simplificar a navegação no projeto.
 - b) acelerem entrega de funcionalidades incrementais, ainda que isso implique acúmulo progressivo de dívidas técnicas no código.
 - c) adotem abordagens flexíveis de implementação, dispensando o uso explícito de princípios de design, desde que os testes automatizados estejam presentes.
 - d) incorporem refatoração contínua, mantenham o código alinhado aos princípios de design e favoreçam a manutenção e a evolução dos sistemas.
 - e) priorizem a consistência transacional entre microsserviços, ainda que isso limite o uso de comunicação assíncrona em arquiteturas orientadas a eventos.

FCC – 2025 – TRT - 15ª Região (SP) – Analista Judiciário - Área Apoio Especializado - Especialidade Tecnologia da Informação

60. O Tribunal Regional do Trabalho está desenvolvendo um sistema de gestão de documentos judiciais que permite o armazenamento, recuperação e edição de diversos tipos de documentos, como petições,

sentenças e relatórios de audiência. O sistema deve permitir que novas funcionalidades sejam adicionadas de forma modular e que alterações nos requisitos de cada tipo de documento não afetem o sistema como um todo. Nessas circunstâncias, a equipe de desenvolvimento deverá aplicar, corretamente, o padrão de projeto mais apropriado, que é o

- a) Singleton.
- b) Adapter.
- c) Factory Method.
- d) Composite.
- e) Decorat.

FCC – 2025 – Prefeitura de São Paulo - SP – Analista de Planejamento e Desenvolvimento Organizacional Tecnologia da Informação e Comunicação

61. A equipe de desenvolvimento de uma prefeitura está refatorando um sistema legado de atendimento ao público e precisa utilizar o padrão Factory Method para criar diferentes tipos de objetos relacionados a solicitações (como solicitações de manutenção, serviços ou emergências). A prática de implementação que reflete adequadamente o padrão Factory Method com foco em extensibilidade e encapsulamento é
- a) criar um método estático para instanciar objetos de solicitações com base em parâmetros fornecidos.
 - b) delegar a criação de objetos para um método de inicialização dentro da própria classe principal.
 - c) criar uma classe base que contenha a lógica para determinar o tipo de solicitação e retornar a instância apropriada.
 - d) centralizar a criação de objetos em uma única classe utilitária que contém métodos estáticos para cada tipo de solicitação.
 - e) utilizar classes abstratas ou interfaces para definir o método de criação e implementá-lo nas subclasses específicas.

FCC – 2025 – Prefeitura de São Paulo - SP – Analista de Planejamento e Desenvolvimento Organizacional Tecnologia da Informação e Comunicação

62. Uma prefeitura está desenvolvendo um sistema para integrar um módulo de pagamentos legados, cujo formato de dados é incompatível com o novo subsistema de cobrança online. É necessário permitir que o novo subsistema utilize o módulo antigo sem modificá-lo diretamente. Nesse contexto, o padrão estrutural Gang of Four (GOF) que resolve de forma ideal o problema de integração entre interfaces incompatíveis é o

	Curso Superior em Análise e Desenvolvimento de Sistemas
	Disciplina: Padrões de Projeto, Frameworks e API
	Profª. Denise do Rocio Maciel
	LISTA DE EXERCÍCIOS: LISTA DE EXERCÍCIO DESIGN PATTERN - ETAPA 2

- a) Adapter, pois converte a interface de uma classe para outra interface esperada pelos clientes, tornando possível a comunicação entre sistemas legados e novos.
- b) Bridge, pois separa a abstração da implementação, permitindo que sistemas legados e novos se comuniquem de forma integrada.
- c) Composite, pois organiza objetos em estruturas hierárquicas, tratando composições e objetos individuais de forma integrada, permitindo a comunicação entre sistemas legados e novos.
- d) Facade, pois fornece uma interface unificada e simplificada para um conjunto complexo de subsistemas, tornando mais simples a comunicação entre sistemas legados e novos.
- e) Flyweight, pois compartilha estado intrínseco para reduzir o consumo de memória quando há múltiplas instâncias semelhantes interagindo entre sistemas ou subsistemas.

FCC – 2023 – TRT - 15ª Região (SP) – Técnico Judiciário - Área de Apoio Especializado - Especialidade Tecnologia da Informação

63. Uma Técnica de um Tribunal Regional do Trabalho utiliza padrões de projeto como soluções típicas para problemas comuns em projeto de software. Ela utilizou um deles que permite que se salve e restaure o estado anterior de um objeto sem revelar os detalhes de sua implementação. Trata-se do padrão

- a) comportamental Memento.
- b) estrutural Mediator.
- c) criacional Singleton.
- d) comportamental Visitor.
- e) estrutural Decorator.

FCC – 2005 – TRE-MG – Analista Judiciário - Análise de Sistemas

64. É um design pattern, inserido no catálogo de padrões de criação (creational) do GoF, que tem por objetivo definir uma interface para criar objetos de forma a deixar subclasses decidirem qual classe instanciar. Também conhecido como Construtor Virtual é o

- a) Prototype.
- b) Factory Method.
- c) Builder.
- d) Singleton.
- e) Evory Abstract.

FCC – 2021 – TJ-SC – Analista de Sistemas

65. Considerando o catálogo GoF. I. Bridge. II. Mediator. III. Proxy. Os padrões de projeto são, respectivamente, do tipo

- a) comportamental, estrutural e comportamental.

- b) criacional, estrutural e comportamental.
- c) estrutural, comportamental e criacional.
- d) estrutural, comportamental e estrutural.
- e) criacional, comportamental e criacional.

FCC – 2020 – AL-AP – Analista Legislativo - Desenvolvedor de Sistemas

66. Sobre a lista de design patterns no padrão GoF, é correto afirmar:

- a) Decorator é um padrão comportamental que categoriza um objeto para fornecer novos comportamentos.
- b) Visitor é um padrão comportamental que encapsula uma solicitação como um objeto.
- c) Abstract Factory é um padrão estrutural que permite criar novas instâncias simplesmente copiando instâncias existentes.
- d) Iterator é um padrão comportamental que fornece uma maneira de acessar sequencialmente uma coleção de objetos sem expor a sua implementação.
- e) State é um padrão criacional que centraliza operações complexas de comunicação e controle entre objetos relacionados.

FCC – 2018 – SEGEF-MA – Analista Executivo - Programador de Sistemas

67. A intenção do padrão de projeto Abstract Factory é

- a) definir uma interface para a criação de um objeto, deixando as subclasses decidirem que classe instanciar. Delega a instanciação para as subclasses.
- b) converter a interface de uma classe na interface esperada pelos clientes. Permite que classes com interfaces incompatíveis trabalhem em conjunto.
- c) fornecer uma interface para a criação de famílias de objetos relacionados ou dependentes sem especificar suas classes completas.
- d) garantir que uma classe tenha somente uma instância e fornecer um ponto global de acesso à mesma.
- e) separar a construção de um objeto complexo de sua representação, de modo que o mesmo processo de construção possa criar diferentes representações.

FCC – 2019 – METRÔ-SP – Analista Desenvolvimento Gestão Júnior – Ciências da Computação

68. Considere os seguintes objetivos de padrões de projeto. 1. Separa a construção de um objeto complexo da sua representação, de forma que o mesmo processo de construção possa criar

	Curso Superior em Análise e Desenvolvimento de Sistemas
	Disciplina: Padrões de Projeto, Frameworks e API
	Profª. Denise do Rocio Maciel
	LISTA DE EXERCÍCIOS: LISTA DE EXERCÍCIO DESIGN PATTERN - ETAPA 2

diferentes representações, ou seja, serve para encapsular a construção de um produto e permitir que ele seja construído em etapas. 2. Atribui responsabilidades adicionais a um objeto dinamicamente. Ele fornece uma alternativa flexível a subclasses para a extensão da funcionalidade, ou seja, envelopa um objeto para fornecer novos comportamentos. 3. Usa compartilhamento para dar suporte a vários objetos de forma eficiente, ou seja, permite que uma instância de uma classe possa ser usada para fornecer muitas 'instâncias virtuais'. 4. Torna possível salvar o estado de um objeto de modo que o mesmo possa ser restaurado, ou seja, permite restaurar um objeto a um dos seus estados prévios, por exemplo, quando o usuário seleciona um 'desfazer'. Correspondem, correta e respectivamente, aos objetivos de padrões de projeto:

- a) Factory Method, Decorator, Facade e Memento.
- b) Adapter, Mediator, Facade e Builder.
- c) Builder, State, Mediator e Adapter.
- d) Builder, Decorator, Flyweight e Memento.
- e) Decorator, Factory Method, Abstract Factory e State.

FCC – 2019 – TJ-MA – Analista Judiciário - Analista de Sistemas - Desenvolvimento

69. O diagrama apresenta a descrição de um design pattern GoF. Esse padrão usa as interfaces genéricas para criar os objetos concretos que fazem parte da família de objetos. O cliente não sabe ou não se importa com quais objetos concretos obtém de cada PatternConcreto, pois usa apenas as interfaces genéricas de seus produtos. Esse design pattern de criação separa detalhes da implementação de um conjunto de objetos de seu uso e é denominado (diagrama UML mostrando SoftwareCliente, interface DesignPattern com createProductA/createProductB, PatternConcretoX/Y, interfaces ProductA/ProductB e suas implementações concretas)

- a) Strategy.
- b) Façade.
- c) Interpreter.
- d) Builder.
- e) Abstract Factory.

FCC – 2019 – SANASA Campinas – Analista de Tecnologia da Informação - Análise e Desenvolvimento

70. Um Analista de TI precisa criar um filtro de requisições em uma aplicação Java EE para processar um conjunto de serviços de maneira padrão. Esse recurso será usado para interceptar e manipular as requisições vindas das páginas web e as respostas retornadas da camada de modelo a estas requisições. O padrão de projeto adequado para esta situação é o

- a) Intercepting Filter.
- b) Filter Chain.
- c) Bridge.
- d) interpreter.
- e) Filter Controller

FCC – 2019 – TRF - 4ª REGIÃO – Analista Judiciário - Sistemas de Tecnologia da Informação

71. Uma Analista de TI quer criar um aplicativo capaz de trocar mensagens entre diversas plataformas móveis, um Android enviando mensagem para um iOS, um Windows Phone trocando mensagens com um Android etc. O problema é que cada uma destas plataformas implementa maneiras diferentes de receber mensagens. Ela sabe que criar vários métodos para cada plataforma não seria uma solução adequada, pois, se o aplicativo precisar incluir uma nova plataforma, outros métodos teriam que ser criados. A modelagem de relacionamentos muitos para muitos pode deixar o projeto do sistema demasiadamente complexo. A Analista procurou, então, um padrão de projeto para utilizar na solução. Dentre os padrões GoF, encontrou um que permite definir um objeto que encapsula a forma como um conjunto de objetos interage. Este padrão promove o acoplamento fraco ao evitar que os objetos se refiram uns aos outros explicitamente e permite variar o uso da interação de forma independente. Atua como um interlocutor entre relacionamentos muitos para muitos, ao evitar uma referência explícita aos objetos, sendo adequado ao problema. Trata-se do padrão comportamental

- a) Mediator.
- b) Composite.
- c) Adapter.
- d) Factory Method.
- e) Prototype.

FCC – 2016 – CREMESP – Analista de Tecnologia da Informação - Análise de Sistemas

72. Suponha que exista uma interface ITraduzFrase que define o método imprimeFrase(). Existem, também, as classes

	Curso Superior em Análise e Desenvolvimento de Sistemas
	Disciplina: Padrões de Projeto, Frameworks e API
	Profª. Denise do Rocio Maciel
	LISTA DE EXERCÍCIOS: LISTA DE EXERCÍCIO DESIGN PATTERN - ETAPA 2

InglesTraduzFrase, EspanholTraduzFrase e AlemãoTraduzFrase que estendem a interface ITraduzFrase e utilizam o método imprimeFrase(). Um Analista utilizou um design pattern que cria objetos de forma dinâmica sem conhecer a classe de implementação. A classe FrasePadrao implementa esse padrão de forma que, de acordo com o idioma desejado, retorna um objeto do tipo ITraduzFrase. O objetivo é que, sendo fornecido o idioma, a frase será nele impressa, em inglês, espanhol ou alemão. Considerando o exemplo e sabendo que a definição do padrão é 'Define uma interface para criar um objeto, mas permite às classes decidirem qual classe instanciar. Este padrão permite a uma classe deferir a instanciação para subclasses.', o nome do padrão usado pelo Analista é

- a) Strategy.
- b) Factory Method.
- c) Singleton.
- d) Chain of Responsibility.
- e) Adapter.

FCC – 2017 – TRF – 5ª REGIÃO – Analista Judiciário - Informática Desenvolvimento

73. O padrão de projeto comportamental que define uma dependência de um-para-muitos entre objetos de modo que, quando um objeto muda de estado, todos os seus dependentes são notificados e atualizados automaticamente é o

- a) Facade.
- b) Observer.
- c) Adapter.
- d) Abstract Factory.
- e) Prototype.

FCC – 2013 – AL-RN – Técnico Legislativo - Programador

74. Em aplicações para web orientadas a objeto, há um design pattern muito utilizado, (relacionado à arquitetura de sistemas) que possui um componente que: – Define o comportamento da aplicação. – Mapeia ações do usuário para classes que cuidam do acesso aos dados. – Seleciona páginas web para entregar resultados vindos das classes de acesso aos dados. O componente citado no texto acima é o

- a) Business.
- b) Controller.
- c) Persistence.
- d) Middle.
- e) Core.

FCC – 2017 – DPE-RS – Analista - Desenvolvimento de Sistemas

75. Considere que é necessário fazer um programa que vá funcionar em várias plataformas, como Windows, Linux, Mac OS etc. O programa fará uso de diversas abstrações de janelas gráficas, por exemplo, janela de diálogo, janela de aviso, janela de erro etc. Um Analista sugeriu o uso de um padrão de projeto GoF estrutural que fornece um nível de abstração maior que o Adapter, pois são separadas as implementações e as abstrações, permitindo que cada uma varie independentemente. O padrão sugerido pelo Analista é o

- a) Bridge e as implementações seriam as classes de janela das plataformas.
- b) Prototype, pois especifica os tipos de plataformas a serem criadas usando uma instância protótipo e cria novas interfaces pela cópia desse protótipo.
- c) MVC, de forma que a camada Model fique responsável por acionar as janelas, a camada Controller exiba os dados do Model ao usuário e a camada View armazene as classes que fazem a comunicação com o Banco de Dados.
- d) Visitor e as interfaces seriam executadas nos elementos de cada plataforma.
- e) State, que permite a uma plataforma alterar o comportamento de sua interface.

FCC – 2017 – ARTESP – Agente de Fiscalização à Regulação de Transporte - Tecnologia de Informação

76. Suponha que em uma organização exista um conjunto de cargos e para cada cargo existem regras de cálculo da taxa de produtividade, cujo valor é determinado por uma porcentagem do salário base do funcionário. Este valor é acrescido ao salário do funcionário como um bônus. Um desenvolvedor deve criar uma solução que utilize um design pattern (padrão de projeto) cuja intenção é: 'definir uma família de algoritmos, encapsular cada uma delas e torná-las intercambiáveis'. Ou seja, o padrão sugere que algoritmos parecidos (métodos de cálculo da taxa de produtividade) sejam separados de quem os utiliza (funcionário). O padrão, além de encapsular os algoritmos da mesma família, também permite a reutilização do código. Nesta situação, caso a regra para cálculo da taxa de produtividade de dois tipos de funcionários, como Desenvolvedor e DBA, seja a mesma, não é necessário escrever código extra. Caso seja necessário incluir um novo cargo, basta implementar sua estratégia de cálculo da taxa ou reutilizar outra similar. Nenhuma outra parte do

	Curso Superior em Análise e Desenvolvimento de Sistemas
	Disciplina: Padrões de Projeto, Frameworks e API
	Profª. Denise do Rocio Maciel
	LISTA DE EXERCÍCIOS: LISTA DE EXERCÍCIO DESIGN PATTERN - ETAPA 2

código precisa ser alterada. O design pattern mencionado na situação apresentada é

- a) Abstract Method.
- b) Builder.
- c) Prototype.
- d) Strategy.
- e) Singleton.

FCC – 2017 – TRE-SP – Técnico Judiciário – Programação de Sistemas

77. Um Programador de Sistemas teve que desenvolver algumas aplicações utilizando padrões de projeto. Sua primeira necessidade foi utilizar um padrão para separar a construção de objetos complexos da sua representação, de forma que o mesmo processo de construção pudesse criar diferentes representações. A segunda necessidade foi o uso de um padrão para permitir que classes incompatíveis trabalhassem em conjunto, já que converte a interface de uma classe em outra, para atender às expectativas do cliente. Para atender suas necessidades, o Programador usou, correta e respectivamente, os padrões de projeto

- a) Factory Method e Singleton.
- b) Builder e Abstract Factory.
- c) Builder e Adapter.
- d) Singleton e Abstract Factory.
- e) Adapter e Bridge.

FCC – 2016 – TRT - 20ª REGIÃO (SE) – Técnico Judiciário - Tecnologia da Informação

78. Um desenvolvedor precisa utilizar um padrão de projeto para interceptar e manipular requisições HTTP de entrada de usuários ao sistema web, e respostas de saída através de filtros de pré-processamento e pós-processamento. Além disso, precisa utilizar outro padrão de projeto capaz de separar as regras de negócio da aplicação das regras de acesso ao banco de dados, permitindo assim centralizar em classes específicas, as operações de conexão ao banco de dados e realização de operações SQL. Os padrões de projeto que o desenvolvedor terá que utilizar são

- a) Front Controller e Data Transfer Object.
- b) Application Filter e Data Session Façade
- c) Intercepting Filter e Data Access Object.
- d) Controller Helper e Data Manager Object.
- e) Application Controller e Data Persistent Object.

FCC – 2016 – Prefeitura de Teresina - PI – Técnico de Nível Superior - Analista de Sistemas

79. Dentre os tipos de Padrões de Projeto (Design Patterns) o que se caracteriza por definir uma interface para a criação de um objeto e que permite que a subclasse decida qual classe instanciar é denominado

- a) Factory Method.
- b) Builder.
- c) Prototype.
- d) Abstract Factory.
- e) Composite.

FCC – 2016 – TRT - 14ª Região (RO e AC) – Analista Judiciário - Tecnologia da Informação

80. Um Analista precisa construir uma aplicação que possui uma função da qual se conhece somente o algoritmo de execução, sendo que o trabalho de codificação para realização da operação desta função tem que ser postergado. Ele optou pelo uso do padrão de projeto comportamental

- a) Template Method, que permite implementar as partes invariantes de um algoritmo e deixar para subclasses a implementação da parte variante.
- b) Adapter, que cria classes reutilizáveis que cooperam com classes não relacionadas ou não previstas.
- c) Singleton, que possibilita o uso de uma instância estendida sem alterar o seu código.
- d) Abstract Factory, que permite criar uma biblioteca de classes de produtos e revelar somente suas interfaces, não suas implementações.
- e) Factory Method, que define uma interface para criar objetos, mas deixa que as subclasses decidam que classe instanciar.

FCC – 2016 – TRT - 14ª Região (RO e AC) – Técnico Judiciário - Tecnologia da Informação

81. Os padrões de projeto

- a) podem deixar um sistema mais complexo ou degradar a sua performance. O seu uso indevido ou inadequado para um determinado contexto constitui-se em um anti pattern.
- b) sempre criam flexibilidade e variabilidade pela introdução de níveis adicionais de endereçamento indireto. Como melhoram o desempenho do sistema devem ser sempre aplicados.
- c) comportamentais abstraem ou adiam o processo de criação dos objetos, ajudando a tornar o sistema dependente de como seus objetos são criados, compostos e representados.
- d) estruturais se concentram nos algoritmos de herança entre os objetos. Eles não descrevem

	Curso Superior em Análise e Desenvolvimento de Sistemas
	Disciplina: Padrões de Projeto, Frameworks e API
	Profª. Denise do Rocio Maciel
	LISTA DE EXERCÍCIOS: LISTA DE EXERCÍCIO DESIGN PATTERN - ETAPA 2

apenas padrões de objetos ou de classes, mas também os padrões de comunicação entre os objetos.

e) de criação se preocupam com a forma como classes e objetos são compostos para formar estruturas maiores. Utilizam o polimorfismo para compor interfaces ou implementações.

FCC – 2016 – TRT - 23ª REGIÃO (MT) – Analista Judiciário - Tecnologia da Informação

82. Um Analista de Sistemas necessita utilizar alguns padrões de projeto para solucionar determinados problemas. De acordo com as Categorias de Padrões do Gang of Four– GoF, ele vai utilizar um padrão de cada uma das três categorias, pela ordem, a saber: – Padrões de Criação – Padrões Estruturais – Padrões Comportamentais. Em relação a essas três categorias, os padrões são, respectivamente,

- a) Strategy, Template Method e Decorator.
- b) Factory Method, Flyweight e Memento.
- c) Bridge, Prototype e Composite.
- d) Abstract Factory, Singleton e Observer
- e) Façade, Proxy e Iterator.

FCC – 2016 – TRT - 23ª REGIÃO (MT) – Técnico Judiciário - Tecnologia da Informação

83. Em muitas situações uma 'aplicação cliente' precisa criar objetos cuja construção efetiva só é definida em tempo de execução. A 'aplicação cliente' não deve se preocupar com a criação dos objetos. Trata-se da motivação do padrão de projeto

- a) Abstract Factory.
- b) Builder.
- c) Factory Method.
- d) Singleton.
- e) Proxy.

FCC – 2016 – TRT - 23ª REGIÃO (MT) – Técnico Judiciário - Tecnologia da Informação

84. Usar compartilhamento para suportar eficientemente grandes quantidades de objeto de granularidade fina é intenção do padrão de projeto

- a) Command.
- b) Composite.
- c) Prototype.
- d) Interpreter.
- e) Flyweight.

FCC – 2015 – DPE-SP – Programador

85. Um Programador está desenvolvendo uma aplicação em que os objetos mudam de

estado com muita frequência. Em tempo de execução, a mudança no estado lógico destes objetos implica também na alteração em seu comportamento. Nessa aplicação é ideal que o Programador utilize um design pattern comportamental cuja classe Context é a interface principal para as requisições dos clientes. O Programador deve utilizar o design pattern

- a) Prototype
- b) State.
- c) Adapter.
- d) Singleton.
- e) Façade.

FCC – 2015 – DPE-RR – Analista de Sistemas

86. Um Analista de Sistemas, em função da extrema tensão causada pelo curto prazo de entrega do novo software, concentrou a maior parte do sistema em uma classe central com dezenas de métodos e atributos. Os métodos foram obtidos de soluções prontas disponíveis nas comunidades de desenvolvedores e ele conseguiu entregar o sistema funcionando no prazo. O Analista

- a) utilizou anti-patterns consagrados para os métodos.
- b) criou a classe central baseado no design pattern Singleton.
- c) utilizou herança ao criar os métodos.
- d) aplicou os princípios do design pattern conhecido como Input Kludge
- e) incorreu em uma solução anti-pattern conhecida como the blob.

FCC – 2015 – TRE-PB – Técnico Judiciário - Área Apoio Especializado - Programação de Sistemas

87. Um técnico deseja usar um padrão de projeto de criação que permita que as subclasses da aplicação possam variar. Este padrão deverá ser focado no processo de instanciação e encapsular a criação de objetos, deixando as subclasses decidirem quais objetos criar e garantindo assim, baixo acoplamento. Para conseguir o que deseja, o técnico selecionou o padrão de projeto que possui uma classe abstrata Creator que define um método específico para criação de objetos. Trata-se do padrão:

- a) Prototype.
- b) Adapter.
- c) Factory Method.
- d) Composite.
- e) Façade.

	Curso Superior em Análise e Desenvolvimento de Sistemas
	Disciplina: Padrões de Projeto, Frameworks e API
	Profª. Denise do Rocio Maciel
	LISTA DE EXERCÍCIOS: LISTA DE EXERCÍCIO DESIGN PATTERN - ETAPA 2

FCC – 2015 – DPE-SP – Analista de Sistemas

88. A empresa em que um Analista trabalha se comprometeu com um cliente em satisfazer todos os requisitos para uma determinada aplicação de software no prazo previsto, porém, de fato, não vai conseguir cumprir o prometido, mesmo assim, vem apresentando funções não implementadas como se já tivessem sido construídas. Percebe-se, nesse caso, o antipattern de gerenciamento de projetos conhecido como

- a) Chain of Responsibility.
- b) Fire Drill.
- c) Abstract Factory.
- d) Smoke and Mirrors.
- e) Analysis Paralysis.

FCC – 2015 – MPE-PB – Analista de Sistemas – Desenvolvedor

89. Considere a seguinte situação: Uma grande empresa possui uma rede de representação de seus produtos constituída por escritórios de venda em diversas cidades. Cada escritório possui um conjunto de vendedores que vão até os varejistas oferecer os produtos. Varejistas podem estar em áreas populosas, como supermercados, ou em áreas remotas, como comércios de bairros rurais. Ao chegar no varejista, o vendedor utiliza um sistema fornecido pela empresa para registrar os pedidos de produtos. Na situação descrita, o objetivo é registrar os pedidos na base de dados da central de distribuição da empresa. Porém, por problemas de comunicação com a central, o computador portátil do varejista pode não conseguir conectar com a base central para registrar os pedidos. Se isso ocorrer, o sistema deve tentar salvá-los na base de dados do escritório. Quando a base do escritório também não estiver acessível, os pedidos deverão ser salvos na base de dados local, no próprio computador portátil do vendedor. Para estruturar a solução para o problema da empresa, o Analista de Sistemas utilizou um padrão de projeto que tem a intenção de evitar o acoplamento do remetente de uma solicitação ao seu receptor, ao dar a mais de um objeto a oportunidade de tratar a solicitação e encadear os objetos receptores, passando a solicitação ao longo da cadeia até que um objeto a trate. O padrão de projeto utilizado é um dos padrões comportamentais da GoF (Gang of Four), denominado

a) Adapter.

- b) Composite Method.
- c) Chain of Responsibility.
- d) Factor Method.
- e) Prototype.

FCC – 2015 – TRT - 3ª Região (MG) – Técnico Judiciário - Área Tecnologia da Informação

90. Considere: I. A aplicação cliente interage com uma fábrica genérica de objetos. II. Os objetos serão gerados efetivamente pela fábrica concreta que estiver sendo utilizada no momento. III. A aplicação cliente não precisa ser configurada para interagir com cada uma das fábricas concretas. IV. Novas fábricas concretas podem ser agregadas, alteradas ou retiradas do sistema sem necessidade de alterações na aplicação cliente. Estas são características e propriedades do padrão de projeto

- a) Factory Method.
- b) Builder
- c) Prototype.
- d) Abstract Factory.
- e) Singleton.

FCC – 2015 – TRT - 15ª Região (SP) – Analista Judiciário – Tecnologia da Informação

91. Os padrões de projeto tornam mais fácil reutilizar projetos e arquiteturas bem sucedidas. Atualmente existem diversos padrões de projetos conforme abaixo: I. Fornece uma interface para a criação de famílias de objetos relacionados ou dependentes sem especificar suas classes concretas. II. Converte a interface de uma classe em outra interface esperada pelos clientes permitindo que certas classes trabalhem em conjunto, pois de outra forma, seria impossível por causa de suas interfaces incompatíveis. III. Fornece uma maneira de acessar sequencialmente os elementos de uma agregação de objetos sem expor sua representação subjacente. Os padrões de projeto apresentados em I, II e III são, respectivamente,

- a) Façade, Builder e Mediator.
- b) Abstract Factory, Adapter e Iterator.
- c) Façade, Adapter e Interpreter.
- d) Singleton, Builder e Mediator.
- e) Abstract Factory, Prototype e Iterator.

FCC – 2015 – CNMP – Analista do CNMP - Tecnologia de Informação e Comunicação Desenvolvimento Sistemas

92. Um Analista de Desenvolvimento de Sistemas do CNMP deve indicar o padrão de projeto mais adequado para ser aplicado na seguinte situação: Uma aplicação que existe

	Curso Superior em Análise e Desenvolvimento de Sistemas
	Disciplina: Padrões de Projeto, Frameworks e API
	Profª. Denise do Rocio Maciel
	LISTA DE EXERCÍCIOS: LISTA DE EXERCÍCIO DESIGN PATTERN - ETAPA 2

simultaneamente em um dispositivo móvel e no ambiente corporativo, necessita de um processo de sincronização entre as informações processadas no dispositivo móvel e na base corporativa. Ambas as aplicações devem se comunicar com um objeto que deve ser único para processar este sincronismo, a fim de evitar a possibilidade de criar dados na base. O padrão de projeto corretamente indicado pelo Analista deve ser

- a) Factory Method, um padrão de criação, que busca definir o fluxo de um algoritmo em uma operação, postergando (deferring) alguns passos para subclasses, sem mudar a estrutura do mesmo.
- b) Prototype, um padrão estrutural, que busca fornecer uma interface para criação de famílias de objetos relacionados ou dependentes sem especificar suas classes concretas.
- c) Singleton, um padrão de criação, que busca garantir que um objeto terá apenas uma única instância, ou seja, uma classe irá gerar apenas um objeto e que este estará disponível de forma única para todo o escopo de uma aplicação.
- d) Command, um padrão comportamental, que busca definir o fluxo de um algoritmo em uma operação, postergando (deferring) alguns passos para subclasses, sem mudar a estrutura do mesmo.
- e) Façade, um padrão estrutural, que busca garantir que um objeto terá apenas uma única instância, ou seja, uma classe irá gerar apenas um objeto e que este estará disponível de forma única para todo o escopo de uma aplicação.

FCC – 2014 – TRT - 13ª Região (PB) – Analista Judiciário - Tecnologia da Informação

93. Angela pretende utilizar alguns design patterns em seu projeto Java e, após algumas pesquisas, encontrou o que buscava em Singleton e Prototype cujos objetivos são, respectivamente: I. Encapsular a escolha das classes concretas a serem utilizadas na criação dos objetos de diversas famílias. II. Permitir a criação de uma única instância de uma classe e fornecer um modo para recuperá-la. III. Possibilitar o reaproveitamento de objetos. IV. Possibilitar a criação de novos objetos a partir da cópia de objetos existentes. Está correto o que consta APENAS em
- a) I e II.
 - b) I e III.
 - c) II e III.

- d) II e IV
- e) III e IV.

FCC – 2014 – TRT - 13ª Região (PB) – Técnico Judiciário - Tecnologia da Informação

94. Os padrões de projeto estruturais diminuem o acoplamento entre os objetos de um sistema orientado a objetos tornando mais simples eventuais alterações no funcionamento do sistema. Um destes padrões permite centralizar todas as requisições a uma aplicação Web, ou seja, haverá um único ponto de entrada na aplicação que receberá e tratará todas as requisições. Este padrão é conhecido como
- a) Bridge.
 - b) Request Controller.
 - c) Manager Request.
 - d) Front Controller.
 - e) Facade.

FCC – 2013 – TRT - 5ª Região (BA) – Técnico Judiciário - Tecnologia da Informação

95. Design patterns podem ser divididos em três categorias: creational, structural e behavioral. Um dos design patterns da categoria creational garante para uma classe específica, apenas uma única instância, a qual é acessível de forma global e uniforme. Este design pattern é conhecido como
- a) Façade.
 - b) Builder.
 - c) Abstract Factory.
 - d) Singleton.
 - e) Composite.

FCC – 2014 – TRT - 16ª REGIÃO (MA) – Técnico Judiciário - Tecnologia da Informação

96. Em relação aos padrões de projeto é INCORRETO afirmar:
- a) São uma porção identificada de conhecimento profundo, que transmite a essência de uma solução provada para um problema recorrente em um certo contexto.
 - b) Descrevem uma estrutura de projeto que resolve um problema particular em um cenário específico e podem ter impacto na maneira pela qual o padrão é aplicado e usado.
 - c) Cada padrão de projeto descreve um problema que ocorre repetidas vezes em um ambiente e, então, descreve o núcleo da solução daquele problema, de tal modo que se pode usar esta solução diversas vezes.
 - d) A descrição de um padrão de projeto pode considerar um conjunto de influências de projeto

	Curso Superior em Análise e Desenvolvimento de Sistemas
	Disciplina: Padrões de Projeto, Frameworks e API
	Profª. Denise do Rocio Maciel
	LISTA DE EXERCÍCIOS: LISTA DE EXERCÍCIO DESIGN PATTERN - ETAPA 2

que descrevem o ambiente e as condições que devem existir para torná-lo aplicável.

e) Padrões arquiteturais e padrões de código são exemplos de padrões de projeto. Os arquiteturais são padrões específicos de uma plataforma e possuem padrões específicos de código, como o Java.

FCC – 2013 – TRT - 15ª Região (SP) – FCC - 2013 - TRT - 15ª Região - Analista Judiciário - Tecnologia da Informação

97. Os padrões Gang of Four (GoF) organizam um conjunto de padrões de projeto (design patterns) em três grupos: de criação, estruturais e comportamentais. Três destes padrões são descritos a seguir: I. Em situações em que classes precisam trabalhar juntas, mas isto não está sendo possível porque suas interfaces são incompatíveis, pode-se utilizar este design pattern que permite converter a interface de uma classe em outra interface esperada pelos clientes de forma que classes com interfaces incompatíveis possam interagir. II. Este design pattern pode ser utilizado quando se deseja definir uma dependência um-para-muitos entre objetos de modo que quando um objeto muda o estado, todos seus dependentes são notificados e atualizados. III. Em situações em que se deseja acessar o conteúdo de uma coleção sem expor sua representação interna utiliza-se este design pattern que permite prover uma interface única para varrer coleções diferentes. Os padrões descritos nos itens I, II e III são, respectivamente,
a) Adapter, Facade e Strategy.
b) Prototype, Composite e Command.
c) Abstract Factory, Observer e Iterator.
d) Adapter, Observer e Iterator.
e) Abstract Factory, Composite e Command.

FCC – 2013 – AL-RN – Analista Legislativo - Analista de Sistemas

98. Analise as seguintes afirmações: I. Fornece uma interface para a criação de uma família de objetos relacionados ou dependentes sem fornecer os detalhes de implementação das classes concretas. II. Converte uma interface de uma classe existente em outra interface esperada pelos clientes. Permite que algumas classes com interfaces diferentes trabalhem em conjunto. III. Separa uma implementação de sua abstração, de forma que ambas possam variar independentemente. IV. Separa a construção de um objeto complexo de sua representação, de modo que o mesmo processo possa criar

representações diferentes. Tratam, respectivamente, dos design patterns:

- a) Builder - Adapter - Bridge - Abstract Factory.
- b) Abstract Factory - Adapter - Bridge - Builder.
- c) Bridge - Adapter - Builder - Abstract Factory.
- d) Adapter - Builder - Abstract Factory - Bridge.
- e) Builder - Bridge - Abstract Factory - Adapter.

FCC – 2013 – DPE-SP – Programador de computador

99. Um design pattern descreve uma solução geral comprovada e reutilizável para um problema recorrente no desenvolvimento de sistemas de software orientados a objetos. Padrões de projeto ajudam a reconhecer e implementar boas soluções para problemas comuns. Dois dos principais design patterns utilizados atualmente são descritos a seguir: I. Visa garantir que uma classe só tenha uma única instância e prover um ponto de acesso global a ela. II. Visa definir uma dependência um-para-muitos entre objetos para que quando um objeto mudar de estado os seus dependentes sejam notificados e atualizados automaticamente. Os design patterns descritos em I e II são, respectivamente:

- a) Singleton e Observer.
- b) Facade e Adapter.
- c) Composite e Adapter.
- d) Singleton e Command.
- e) Facade e Observer.

FCC – 2011 – TCE-PR – Analista de Controle - Informática

100. Os design patterns
a) são projetos de arquitetura para um domínio específico de aplicação e sempre trazem componentes predefinidos que envolvem código de programação.
b) consistem em conjuntos de classes que um usuário instancia para utilizar seus métodos. Após a chamada ao método, o controle do fluxo da aplicação retorna para o usuário.
c) são de uso exclusivo em processos de desenvolvimento de soluções orientadas a objetos, já que os objetos são a mais adequada abstração para o reuso.
d) são aplicações propriamente ditas, normalmente construídas pela integração de diversos frameworks.
e) podem ser modelados utilizando-se a linguagem UML que fornece um meio eficiente de modelar padrões de projeto representando-os como colaborações.

FCC – 2011 – TRE-RN – Programador de computador

	Curso Superior em Análise e Desenvolvimento de Sistemas
	Disciplina: Padrões de Projeto, Frameworks e API
	Profª. Denise do Rocio Maciel
	LISTA DE EXERCÍCIOS: LISTA DE EXERCÍCIO DESIGN PATTERN - ETAPA 2

101. Na engenharia de software, os padrões de projetos comportamentais tratam das interações e divisões de responsabilidades entre as classes ou objetos. São exemplos típicos dessa família:

- Cammand, Factory Method e Prototype.
- Builder, Prototype e Singleton.
- Chain of Responsibility, Interpreter e Iterator.
- Adapter, Bridge e Façade.
- Abstract Factory, Builder e Composite.

FCC – 2011 – TRT - 14ª Região (RO e AC) – Analista Judiciário - Tecnologia da Informação

102. No contexto dos padrões de projeto: I. Oferecer uma interface simples para uma coleção de classes. II. Desacoplar uma abstração de sua implementação para que ambas possam variar independentemente. Correspondem

- respectivamente a
- Façade e Bridge.
- Adapter e Façade.
- Composite e Bridge.
- Façade e Composite.
- Bridge e Adapter.

FCC – 2011 – TRT - 4ª REGIÃO (RS) – Analista Judiciário - Tecnologia da Informação

103. O catálogo de padrões de projeto (design patterns) do GoF contém

- 20 padrões e está basicamente dividido em duas seções: Structural e Behavioral.
- 21 padrões e está basicamente dividido em duas seções: Creational e Behavioral.
- 23 padrões e está basicamente dividido em duas seções: Structural e Behavioral.
- 23 padrões e está, basicamente, dividido em três seções: Creational, Structural e Behavioral.
- 24 padrões e está basicamente dividido em três seções: Creational, Spectral e Behavioral.

FCC – 2011 – TRT - 24ª REGIÃO (MS) – Analista Judiciário - Tecnologia da Informação

104. Considere: I. Fornecer uma interface para criação de famílias de objetos relacionados ou dependentes, sem especificar suas classes concretas. Possibilitar o adiamento da instanciação para as subclasses. II. Garantir a existência de apenas uma instância de uma classe, mantendo um ponto global de acesso ao seu objeto. III. Possibilitar o armazenamento do estado interno de um objeto em um determinado momento, para que seja possível retorná-lo a esse estado, caso necessário. I, II e III são,

respectivamente, objetivos dos design patterns intitulados:

- Interpreter, Iterator e Memento.
- Command, Singleton e Iterator.
- Factory Method, Singleton e Memento.
- Iterator, Factory Method e Flyweight.
- Singleton, Flyweight e Command.

GABARITO

- selecionar uma capacidade do legado, implementar sua nova versão em serviço separado e encaminhar gradualmente o tráfego correspondente para essa implementação.
- Factory Method, Singleton e Observer.
- Builder, que separa a construção de um objeto complexo de sua representação, permitindo que o mesmo processo de construção crie diferentes representações.
- incorporem refatoração contínua, mantenham o código alinhado aos princípios de design e favoreçam a manutenção e a evolução dos sistemas.
- Factory Method.
- utilizar classes abstratas ou interfaces para definir o método de criação e implementá-lo nas subclasses específicas.
- Adapter, pois converte a interface de uma classe para outra interface esperada pelos clientes, tornando possível a comunicação entre sistemas legados e novos.
- comportamental Memento.
- Factory Method.
- estrutural, comportamental e estrutural.
- Iterator é um padrão comportamental que fornece uma maneira de acessar sequencialmente uma coleção de objetos sem expor a sua implementação.
- fornecer uma interface para a criação de famílias de objetos relacionados ou dependentes sem especificar suas classes completas.
- Builder, Decorator, Flyweight e Memento.
- Abstract Factory.
- Intercepting Filter.
- Mediator.
- Factory Method.
- Observer.
- Controller.
- Bridge e as implementações seriam as classes de janela das plataformas.
- Strategy.
- Builder e Adapter.
- Intercepting Filter e Data Access Object.

	Curso Superior em Análise e Desenvolvimento de Sistemas
	Disciplina: Padrões de Projeto, Frameworks e API
	Profª. Denise do Rocio Maciel
	LISTA DE EXERCÍCIOS: LISTA DE EXERCÍCIO DESIGN PATTERN - ETAPA 2

24. Factory Method.
25. Template Method, que permite implementar as partes invariantes de um algoritmo e deixar para subclasses a implementação da parte variante.
26. podem deixar um sistema mais complexo ou degradar a sua performance. O seu uso indevido ou inadequado para um determinado contexto constitui-se em um anti pattern.
27. Factory Method, Flyweight e Memento.
28. Factory Method.
29. Flyweight.
30. State.
31. incorreu em uma solução anti-pattern conhecida como the blob.
32. Factory Method.
33. Smoke and Mirrors.
34. Chain of Responsibility.
35. Abstract Factory.
36. Abstract Factory, Adapter e Iterator.
37. Singleton, um padrão de criação, que busca garantir que um objeto terá apenas uma única instância, ou seja, uma classe irá gerar apenas um objeto e que este estará disponível de forma única para todo o escopo de uma aplicação.
38. II e IV
39. Front Controller.
40. Singleton.
41. Padrões arquiteturais e padrões de código são exemplos de padrões de projeto. Os arquiteturais são padrões específicos de uma plataforma e possuem padrões específicos de código, como o Java.
42. Adapter, Observer e Iterator.
43. Abstract Factory - Adapter - Bridge - Builder.
44. Singleton e Observer.
45. podem ser modelados utilizando-se a linguagem UML que fornece um meio eficiente de modelar padrões de projeto representando-os como colaborações.
46. Chain of Responsibility, Interpreter e Iterator.
47. Façade e Bridge.
48. 23 padrões e está, basicamente, dividido em três seções: Creational, Structural e Behavioral.
49. Factory Method, Singleton e Memento.
50. Builder pattern.
51. I e III, apenas.
52. Singleton, Adapter, Command e Controller.
53. Prototype, Façade, Iterator e Expert.
54. O padrão é uma descrição de conhecimento e experiência acumulados, uma solução comprovada para um problema comum.
55. Model-View-Controller.
56. selecionar uma capacidade do legado, implementar sua nova versão em serviço separado e encaminhar gradualmente o tráfego correspondente para essa implementação.
57. Factory Method, Singleton e Observer.
58. Builder, que separa a construção de um objeto complexo de sua representação, permitindo que o mesmo processo de construção crie diferentes representações.
59. incorporem refatoração contínua, mantenham o código alinhado aos princípios de design e favoreçam a manutenção e a evolução dos sistemas.
60. Factory Method.
61. utilizar classes abstratas ou interfaces para definir o método de criação e implementá-lo nas subclasses específicas.
62. Adapter, pois converte a interface de uma classe para outra interface esperada pelos clientes, tornando possível a comunicação entre sistemas legados e novos.
63. comportamental Memento.
64. Factory Method.
65. estrutural, comportamental e estrutural.
66. Iterator é um padrão comportamental que fornece uma maneira de acessar sequencialmente uma coleção de objetos sem expor a sua implementação.
67. fornecer uma interface para a criação de famílias de objetos relacionados ou dependentes sem especificar suas classes completas.
68. Builder, Decorator, Flyweight e Memento.
69. Abstract Factory.
70. Intercepting Filter.
71. Mediator.
72. Factory Method.
73. Observer.
74. Controller.
75. Bridge e as implementações seriam as classes de janela das plataformas.
76. Strategy.
77. Builder e Adapter.
78. Intercepting Filter e Data Access Object.
79. Factory Method.
80. Template Method, que permite implementar as partes invariantes de um algoritmo e deixar para subclasses a implementação da parte variante.
81. podem deixar um sistema mais complexo ou degradar a sua performance. O seu uso indevido

	Curso Superior em Análise e Desenvolvimento de Sistemas
	Disciplina: Padrões de Projeto, Frameworks e API
	Profª. Denise do Rocio Maciel
	LISTA DE EXERCÍCIOS: LISTA DE EXERCÍCIO DESIGN PATTERN - ETAPA 2

ou inadequado para um determinado contexto constitui-se em um anti pattern.

82. Factory Method, Flyweight e Memento.

83. Factory Method.

84. Flyweight.

85. State.

86. incorreu em uma solução anti-pattern conhecida como the blob.

87. Factory Method.

88. Smoke and Mirrors.

89. Chain of Responsibility.

90. Abstract Factory.

91. Abstract Factory, Adapter e Iterator.

92. Singleton, um padrão de criação, que busca garantir que um objeto terá apenas uma única instância, ou seja, uma classe irá gerar apenas um objeto e que este estará disponível de forma única para todo o escopo de uma aplicação.

93. II e IV

94. Front Controller.

95. Singleton.

96. Padrões arquiteturais e padrões de código são exemplos de padrões de projeto. Os arquiteturais são padrões específicos de uma plataforma e possuem padrões específicos de código, como o Java.

97. Adapter, Observer e Iterator.

98. Abstract Factory - Adapter - Bridge - Builder.

99. Singleton e Observer.

100. podem ser modelados utilizando-se a linguagem UML que fornece um meio eficiente de modelar padrões de projeto representando-os como colaborações.

101. Chain of Responsibility, Interpreter e Iterator.

102. Façade e Bridge.

103. 23 padrões e está, basicamente, dividido em três seções: Creational, Structural e Behavioral.

104. Factory Method, Singleton e Memento.